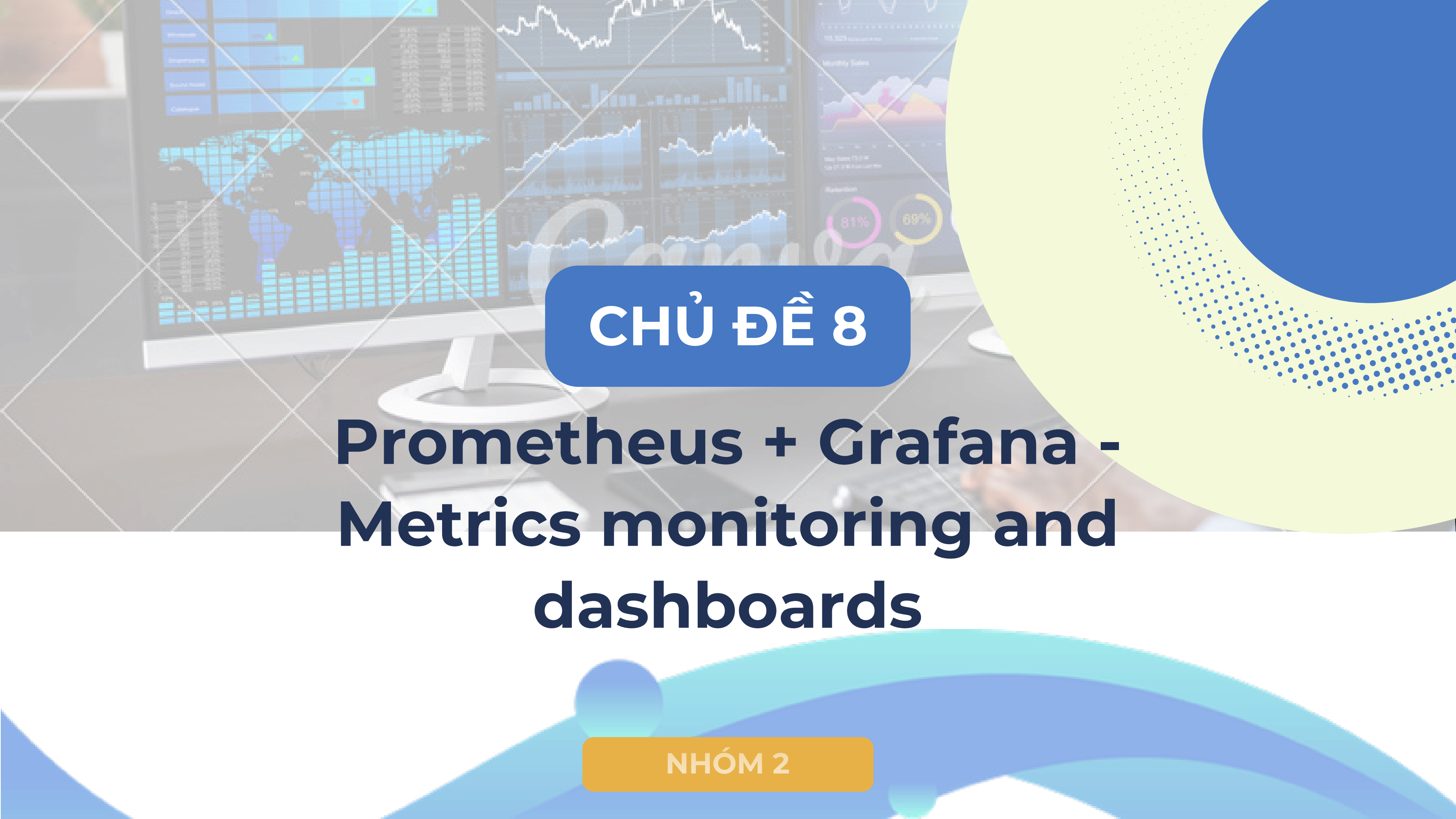
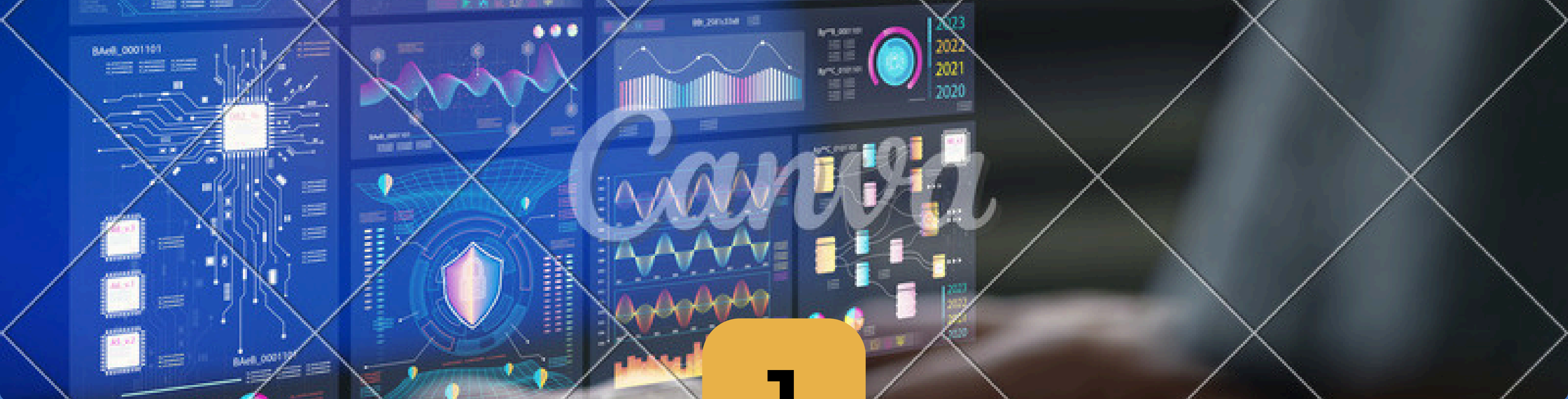




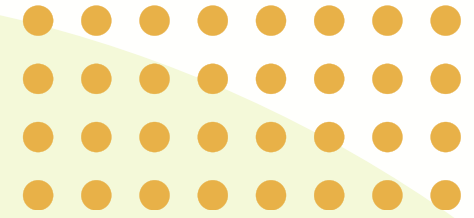
CHỦ ĐỀ 8

Prometheus + Grafana - Metrics monitoring and dashboards

NHÓM 2



Giới thiệu



1.1 Monitoring là gì?

Quá trình theo dõi liên tục trạng thái hệ thống, server, ứng dụng qua các metric: CPU, RAM, disk, network, response time

Mục tiêu: phát hiện sớm bất thường, xử lý kịp thời trước khi ảnh hưởng người dùng

Nếu không có monitoring, quản trị viên thường chỉ biết sự cố khi user báo lỗi (bị động)



1.2 Prometheus là gì?



Hệ thống giám sát và cảnh báo mã nguồn mở, lưu trữ dữ liệu metric dưới dạng time-series



Ra đời tại SoundCloud năm 2012, hiện là dự án tiêu biểu của CNCF

1.2 Prometheus là gì?



Vai trò chính:

- Thu thập metric qua cơ chế pull-based (scrapping) từ các target
- Lưu trữ dữ liệu metric theo thời gian để theo dõi xu hướng
- Truy vấn dữ liệu bằng ngôn ngữ PromQL linh hoạt
- Hỗ trợ xây dựng cảnh báo khi hệ thống vượt ngưỡng hoặc bất thường

1.3 Grafana là gì?



- > **Grafana là một nền tảng mã nguồn mở dùng để trực quan hóa dữ liệu. Mạnh trong việc xây dựng các dashboard theo dõi hệ thống**
- > **Phát triển lần đầu bởi Torkel Ödegaard năm 2014, phổ biến trong observability**



1.3 Grafana là gì?



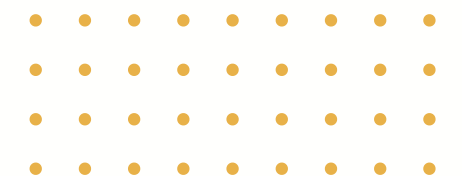
Vai trò chính của Grafana:

- > Kết nối nhiều nguồn dữ liệu khác nhau, trong đó có Prometheus
- > Hiển thị dữ liệu dưới dạng biểu đồ, bảng, gauge, dashboard tổng hợp
- > Giúp quan sát xu hướng thay đổi hệ thống theo thời gian dễ dàng hơn
- > Hỗ trợ cảnh báo trực quan và chia sẻ dashboard cho nhóm vận hành

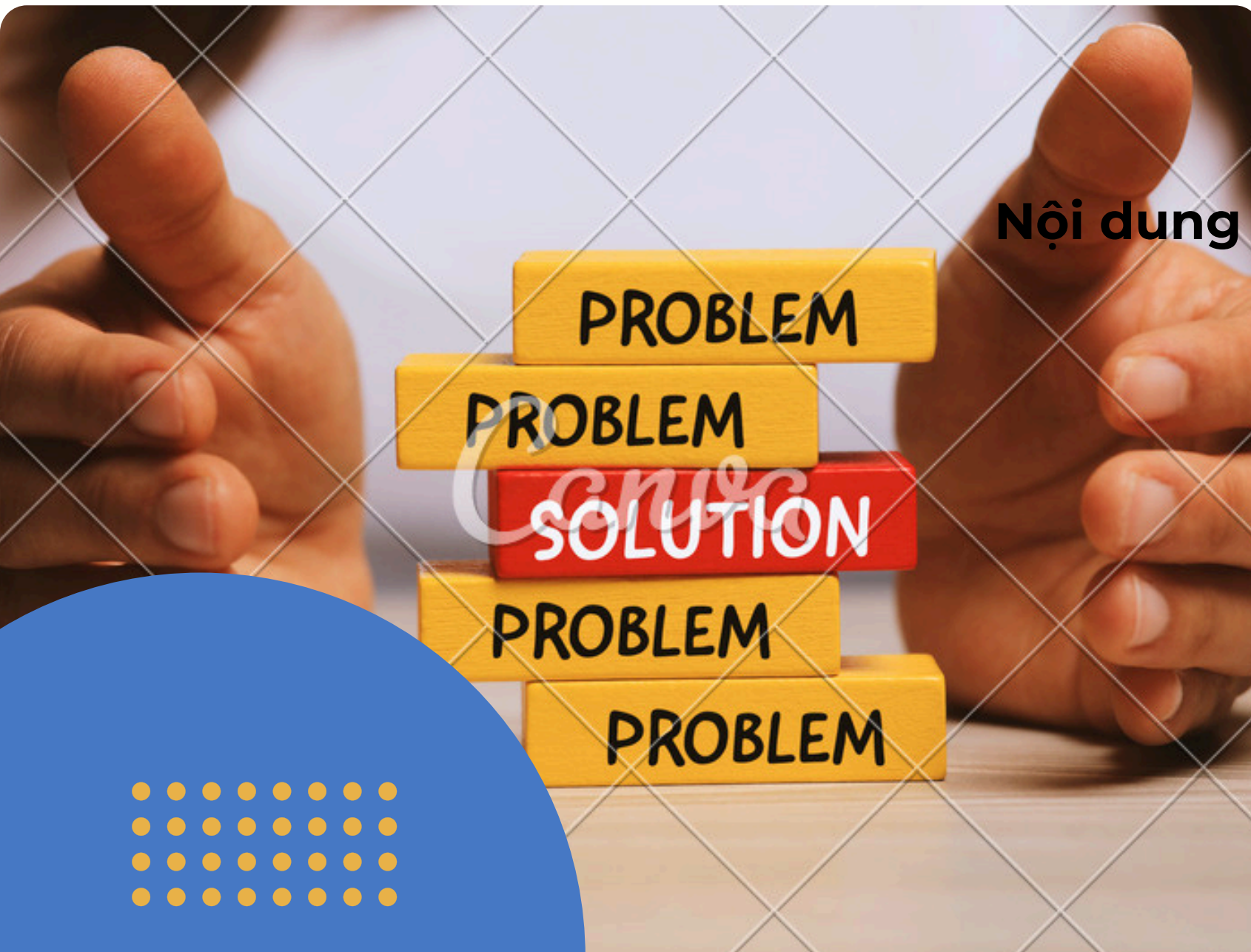


1.4 Grafana vs Prometheus

	 Prometheus	 Grafana
 Vai trò	 Thu thập và lưu trữ metric theo thời gian	 Trực quan hóa dữ liệu thành dashboard
 Input	 Scrape metric định kỳ từ các target	 Truy vấn dữ liệu từ Prometheus (data source)
 Output	 Time-series data, truy vấn bằng PromQL	 Biểu đồ, bảng, cảnh báo trực quan
 Lưu trữ	 Time-series database (TSDB) cục bộ	 Không lưu trữ lâu dài, chỉ lưu cấu hình dashboard và data source
 Ngôn ngữ truy vấn	 PromQL	 SQL Sử dụng PromQL thông qua query editor
 Cảnh báo	 Hỗ trợ Alerting & Alertmanager	 Hỗ trợ cảnh báo trực quan dựa trên dữ liệu từ data source
 Triển khai	 Thường chạy như một service độc lập	 Thường chạy như một service độc lập



1.5 Vấn đề thực tế khi thiếu monitoring



QTV có thể gặp nhiều vấn đề KHÔNG phát hiện kịp thời:

- Website phản hồi chậm vào giờ cao điểm do CPU hoặc RAM tăng cao
- Ổ đĩa gần đầy vì log và dữ liệu ứng dụng tăng nhanh theo thời gian
- Service quan trọng như Nginx, database, backend dừng nhưng chưa xử lý kịp
- Lưu lượng truy cập tăng đột biến, container hoặc service liên tục restart
- Ứng dụng vẫn chạy nhưng xử lý request chậm do lỗi database hoặc kết nối mạng

1.7 Cách Prometheus + Grafana giải quyết

- Prometheus định kỳ thu thập metric: CPU, RAM, disk, network, trạng thái service
- Dữ liệu lưu theo thời gian → quan sát được xu hướng thay đổi, không chỉ tình trạng hiện tại
- Grafana kết nối và trực quan hóa dữ liệu → dễ nhận ra bất thường hơn đọc log thủ công
- Cấu hình cảnh báo khi vượt ngưỡng: CPU trên 80%, RAM/disk gần đầy, target chuyển DOWN



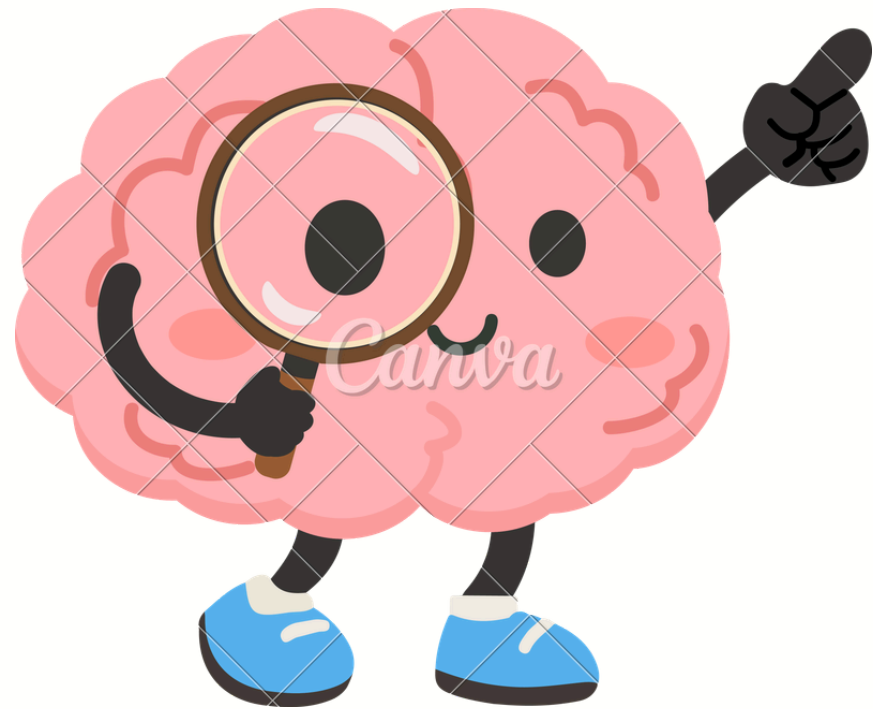


2

Thuật ngữ và khái niệm cốt lõi

2.1 Metric là gì?

- Định nghĩa: Metric là các chỉ số định lượng đo lường trạng thái, hiệu năng hoặc hành vi của một hệ thống, ứng dụng tại một thời điểm cụ thể. Mỗi metric thường bao gồm một tên gọi và một giá trị số.



- Ví dụ đi kèm:
 - **node_memory_Active_bytes**: Lượng bộ nhớ RAM đang hoạt động.
 - **http_requests_total**: Tổng số lượng request HTTP đã gửi đến hệ thống backend.

2.2 Time-series là gì?

- Định nghĩa: là tập hợp các điểm dữ liệu của cùng một metric được ghi nhận và sắp xếp liên tiếp theo trình tự thời gian.
- Mỗi điểm dữ liệu trong chuỗi thời gian là một cặp giá trị gồm: [Timestamp - Value].
- Trong Prometheus, các chuỗi thời gian này còn được phân loại chi tiết nhờ các nhãn (labels) đi kèm dạng Key-Value.
- Ví dụ đi kèm: Metric `node_cpu_seconds_total{cpu="0", mode="system"}` lưu chuỗi dữ liệu:
 - 1717000000 -> 12.5 (lúc 10:00:00)
 - 1717000005 -> 14.2 (lúc 10:00:05)
 - 1717000010 -> 16.1 (lúc 10:00:10)



2.3 Exporter và node_exporter

Exporter: là một công cụ trung gian chạy độc lập, có nhiệm vụ thu thập các chỉ số thô từ một đối tượng cụ thể (hệ điều hành, database, web server), chuyển đổi chúng sang định dạng chuẩn của Prometheus và expose ra qua một cổng HTTP (endpoint /metrics).

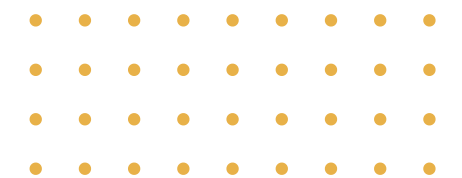
node_exporter: là một exporter chính thức do cộng đồng Prometheus phát triển, chuyên dùng để thu thập các chỉ số phần cứng và tài nguyên hệ điều hành của các máy chủ Linux/Unix.

- Ví dụ đi kèm: khi bạn cài node_exporter lên một máy chủ Linux và mở cổng `http://<IP_SERVER>:9100/metrics`, nó sẽ trả về một trang văn bản thô chứa toàn bộ thông số về dung lượng ổ đĩa, CPU, RAM và băng thông mạng của máy chủ đó để Prometheus đến lấy.

2.4 Scraping là gì?

Định nghĩa: scraping (hoặc Poll/Scrape) là hành động Prometheus chủ động gửi một request HTTP GET đến endpoint /metrics của các Target theo một chu kỳ thời gian được thiết lập sẵn để kéo toàn bộ dữ liệu metric về lưu trữ.

Ví dụ đi kèm: cấu hình scrape_interval: 15s trong Prometheus nghĩa là cứ đúng 15 giây, Prometheus sẽ tự động "gõ cửa" node_exporter một lần để lấy toàn bộ chỉ số CPU/RAM mới nhất về máy.



2.5 Dashboard

- Định nghĩa: dashboard là một bảng điều khiển giao diện đồ họa tổng hợp, nơi chứa nhiều biểu đồ, đồ thị, bảng biểu được sắp xếp trực quan trên cùng một màn hình.
- Dashboard giúp người quản trị có cái nhìn toàn diện về "sức khỏe" hệ thống trong một không gian duy nhất mà không cần đọc log thủ công.
 - Ví dụ đi kèm: một Dashboard giám sát Linux Server trên Grafana sẽ bao gồm: 1 biểu đồ đường (Time-series) chạy mượt mà hiển thị phần trăm CPU, 1 đồng hồ tròn (Gauge) hiển thị RAM đang dùng và một ô trạng thái (Stat) hiện màu xanh thể hiện server đang UP.



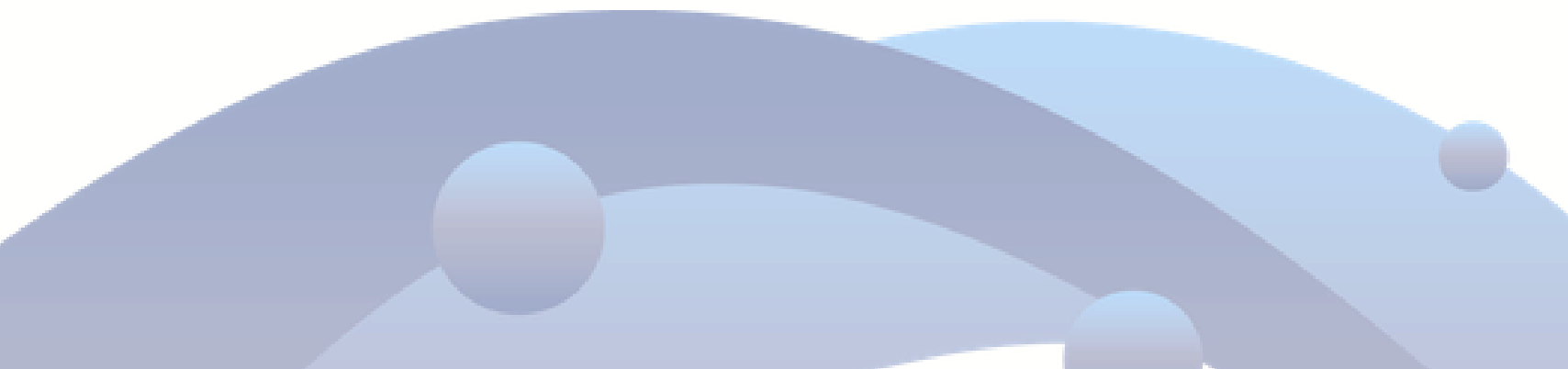
2.6 Target và job

- Target là một đối tượng cụ thể mà Prometheus đã được cấu hình để kết nối và thực hiện hành động scraping dữ liệu metric.
- Target: 192.168.1.50:9100 (địa chỉ của Node Exporter trên Server A) hoặc localhost:9090 (chính cổng nội bộ của Prometheus).

- Job là một tập hợp hoặc một nhóm các Target có cùng bản chất, cùng tính năng được gộp lại với nhau để Prometheus dễ quản lý và phân loại dữ liệu khi thu thập.
- Job: Bạn có 3 server chạy web backend giống hệt nhau, bạn gộp chúng vào một job tên là job: 'backend-services'. Khi đó Prometheus sẽ hiểu cả 3 địa chỉ IP này đều thuộc nhóm xử lý backend.

2.7 Data source là gì?

- **Định nghĩa:** Data Source là nguồn cấp dữ liệu. Đối với Grafana, bản thân nó không lưu trữ dữ liệu, nên nó cần bạn khai báo kết nối tới nơi đang lưu dữ liệu thô để nó có thể kéo về vẽ biểu đồ.
- **Ví dụ đi kèm:** Khi bạn cấu hình Grafana, bạn chọn loại Data Source là Prometheus và điền URL là `http://localhost:9090`. Lúc này Grafana đã kết nối thành công với kho dữ liệu của Prometheus.



2.8 PromQL là gì?

- Định nghĩa: là ngôn ngữ truy vấn chuyên dụng được thiết kế riêng cho Prometheus. Nó cho phép người dùng lọc, tính toán, tổng hợp và xử lý các dữ liệu chuỗi thời gian (Time-series) theo thời gian thực để phục vụ cho việc hiển thị đồ thị hoặc kích hoạt cảnh báo.
- Ví dụ đi kèm: “`100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`”: Câu lệnh PromQL kinh điển dùng để tính toán chính xác tỷ lệ phần trăm CPU bận trung bình trong vòng 5 phút vừa qua của các server.





3

Kiến trúc và cách hoạt động

Luồng dữ liệu Prometheus và Grafana



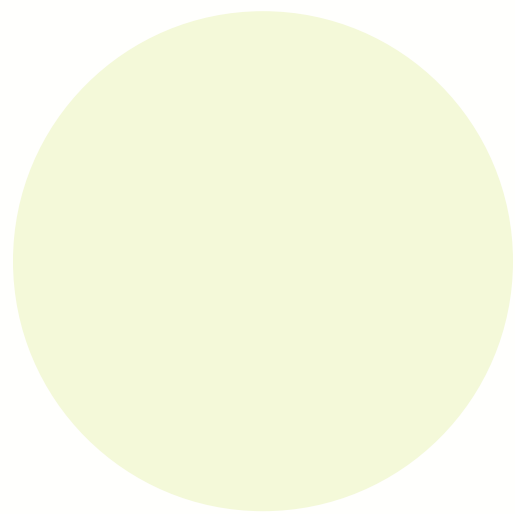


- **node_exporter chạy trên Linux VM:** đây là một agent nhẹ, có nhiệm vụ thu thập các chỉ số phần cứng và hệ điều hành (CPU, RAM, disk, network...) rồi expose ra dưới dạng plain-text tại endpoint `/metrics` trên cổng mặc định 9100. node_exporter hoàn toàn thụ động — nó chỉ "đứng chờ" để bị đọc, không tự gửi dữ liệu đi đâu cả.
- **Prometheus Server chủ động scrape (pull):** theo cấu hình `scrape_configs`, Prometheus định kỳ (ví dụ mỗi 15s) gửi HTTP request đến endpoint `/metrics` của từng target đã khai báo, lấy dữ liệu về và lưu thành các time series có timestamp vào TSDB (Time Series Database) tích hợp sẵn, lưu trên local disk của chính Prometheus.
- **Grafana truy vấn Prometheus để hiển thị:** Grafana được cấu hình Prometheus làm data source. Khi người dùng mở dashboard, Grafana gửi các câu query PromQL đến Prometheus, nhận về dữ liệu time series và render thành biểu đồ/panel theo thời gian thực.

Pull khác gì so với Push

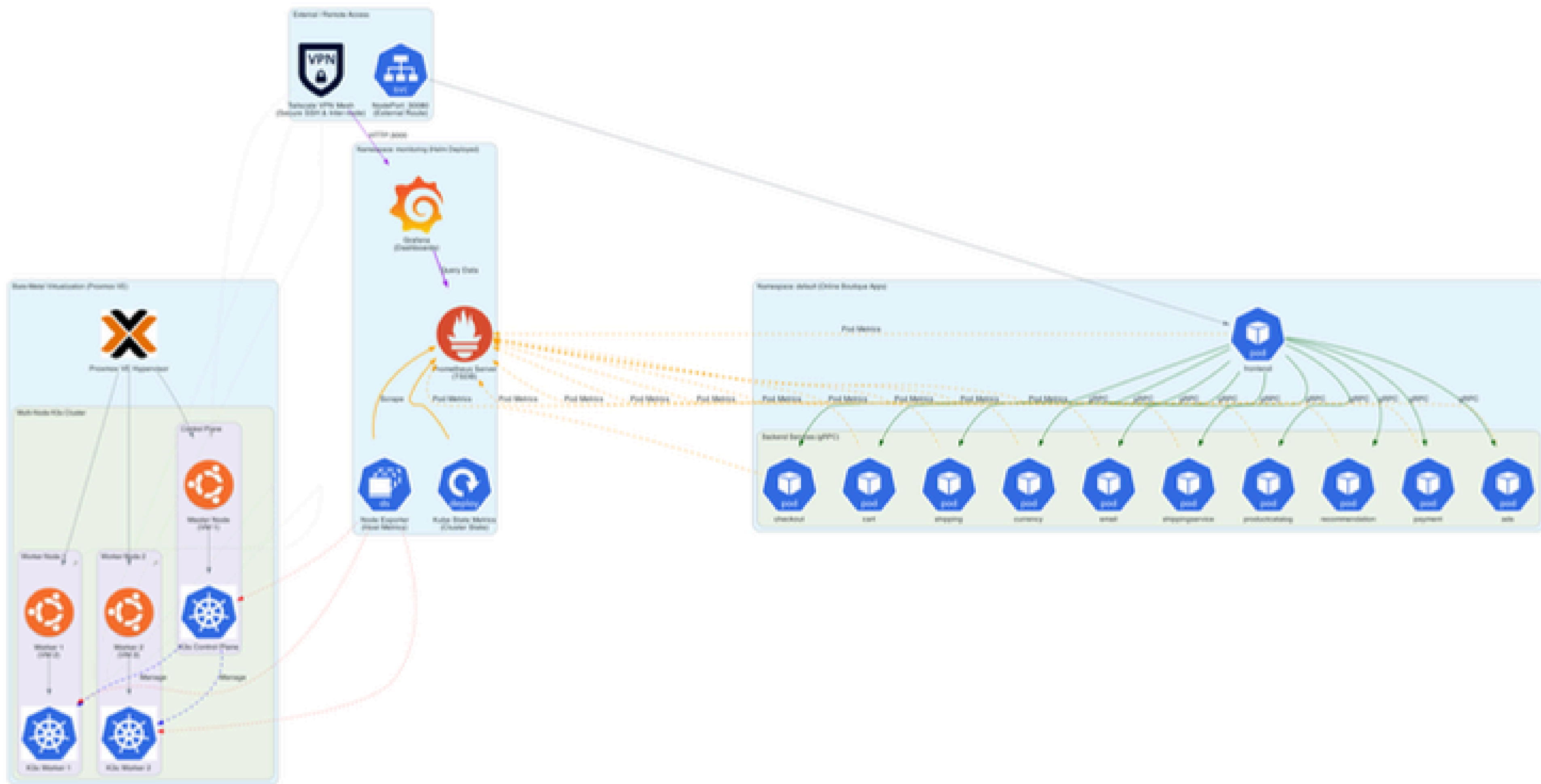
Push: target (ứng dụng, agent) chủ động gửi (đẩy) metric của mình đến một server tập trung theo chu kỳ hoặc theo sự kiện. Target biết địa chỉ của server, tự mở kết nối, tự quyết định khi nào gửi

Pull: server chủ động đi "kéo" (scrape/lấy) dữ liệu từ target theo chu kỳ mà server tự định. Target chỉ cần expose sẵn dữ liệu ở một endpoint (ví dụ /metrics), không cần biết gì về server, không chủ động gửi đi đâu cả. Đây là mô hình của Prometheus.



Prometheus chọn pull-based ? Why ?

- **Dễ kiểm soát tập trung:** server quyết định scrape target nào, tần suất bao nhiêu — không phụ thuộc vào việc từng agent tự cấu hình đúng địa chỉ đích, giảm rủi ro cấu hình sai gây quá tải server.
- **Tự động phát hiện target "chết":** nếu một lần scrape thất bại (timeout, connection refused...), Prometheus tự ghi nhận metric up = 0 cho target đó — cơ chế health-check gần như miễn phí, không cần logic riêng.
- **An toàn hơn về mặt mạng:** target chỉ cần mở một endpoint HTTP để bị đọc, không cần biết địa chỉ server và tự thiết lập kết nối ra ngoài — dễ kiểm soát bằng firewall/security group hơn.
- **Phù hợp môi trường có service discovery:** Prometheus có thể tự động phát hiện target mới (qua file_sd, Kubernetes SD...) và bắt đầu scrape ngay, không cần cấu hình lại phía target.





4

Ứng dụng thực tế

04. ỨNG DỤNG THỰC TẾ

GIÁM SÁT HẠ TẦNG PHẦN CỨNG (HARDWARE MONITORING)

CHỈ SỐ ĐO & GIẢI PHÁP KỸ THUẬT

MÔ HÌNH THU THẬP:



CPU **Tỷ lệ % CPU Busy**
Đo lường chi tiết thời gian CPU thực tế xử lý các tác vụ hệ thống.

RAM **Bộ nhớ RAM sử dụng**
Tỷ lệ dung lượng RAM thực tế đang dùng so với tổng RAM máy chủ.

DISK **Dung lượng đĩa phân vùng root (/)**
Giám sát dung lượng ổ cứng còn trống để đảm bảo không gian lưu trữ.

GIÁ TRỊ THỰC TIỄN ĐEM LẠI

🎯 **Giám sát tập trung**

Theo dõi liên tục và toàn diện "sức khỏe" hạ tầng máy chủ tại một giao diện duy nhất, loại bỏ hoàn toàn việc SSH thủ công vào từng node để kiểm tra.

⚠️ **Cảnh báo chủ động**

Phát hiện sớm các dấu hiệu bất thường như **Rò rỉ bộ nhớ (Memory Leak)** hoặc nguy cơ **cạn kiệt ổ đĩa** giúp xử lý kịp thời trước khi hệ thống xảy ra sự cố.

04. ỨNG DỤNG THỰC TẾ

GIÁM SÁT TẦNG ỨNG DỤNG & WEB SERVER

GIẢI PHÁP & 3 TÍN HIỆU VÀNG (GOLDEN SIGNALS)

GIẢI PHÁP TRIỂN KHAI:

nginx-prometheus-exporter → Prometheus → Grafana

RPS

Throughput (Số yêu cầu / giây)

Tốc độ xử lý yêu cầu (Requests Per Second) của Web Server.

AC

Active Connections (Kết nối hoạt động)

Số lượng kết nối đang mở để đánh giá độ nghẽn mạng thời gian thực.

ERR

Error Rate (Tỷ lệ mã lỗi HTTP)

Giám sát tỷ lệ phản hồi các mã lỗi HTTP (4xx / 5xx) từ ứng dụng.

GIÁ TRỊ THỰC TIỄN ĐEM LẠI

Góc nhìn Người dùng cuối

Đánh giá chính xác hiệu năng hoạt động của ứng dụng dưới góc nhìn thực tế của người dùng cuối thay vì chỉ theo dõi chỉ số phần cứng thô.

Phát hiện & Cô lập sự cố

Phát hiện tức thì ngay khi dịch vụ Web bị sập hoặc gặp lỗi mã nguồn trên diện rộng → Cô lập vùng lỗi nhanh chóng và giảm tối đa thời gian gián đoạn dịch vụ (Downtime).



5

Giới thiệu sơ lược về Zabbix

Tổng quan & Kiến trúc **Zabbix**

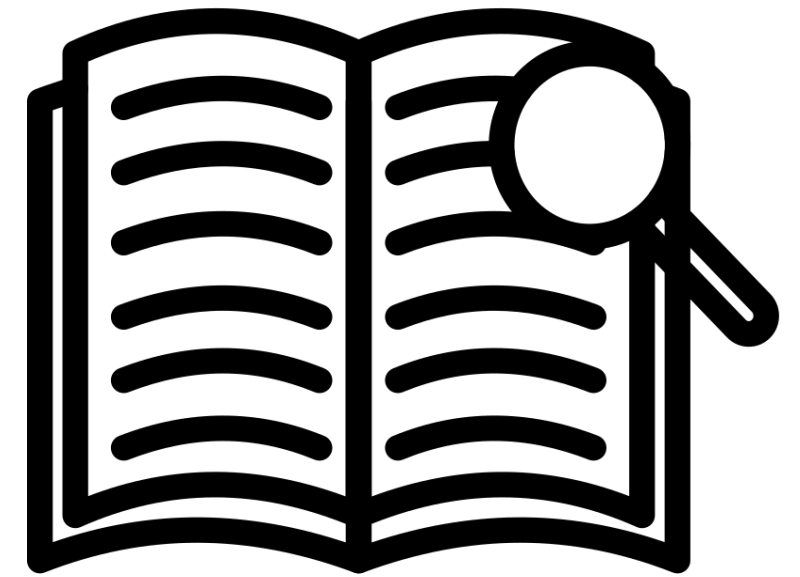
Zabbix là nền tảng giám sát "All-in-one" truyền thống

-  Theo dõi trạng thái hoạt động của các máy chủ và dịch vụ
-  Giám sát hiệu năng sử dụng CPU, RAM, ổ cứng và băng thông mạng.
-  Phát hiện sớm các lỗi hoặc dấu hiệu bất thường và thông báo
-  Hỗ trợ phân tích xu hướng hoạt động của hệ thống thông qua dữ liệu lịch sử.



Thành phần của **Zabbix**

- **Zabbix Server:** Thành phần trung tâm, tiếp nhận và xử lý dữ liệu, đánh giá Trigger, tạo Problem/Event và thực hiện cảnh báo.
- **Zabbix Agent:** Cài trên máy cần giám sát, thu thập CPU, RAM, Disk, Network, Process, Log và trao đổi dữ liệu với Server/Proxy.
- **Zabbix Proxy:** Trung gian thu thập dữ liệu tại các mạng/chi nhánh từ xa, giúp giảm tải Server và lưu tạm dữ liệu khi mất kết nối.
- **Database:** Lưu trữ cấu hình, History, Trends, Events và Problems của hệ thống.
- **Web Frontend:** Giao diện web để cấu hình, quản lý và trực quan hóa dữ liệu monitoring.



Nguyên lí hoạt động

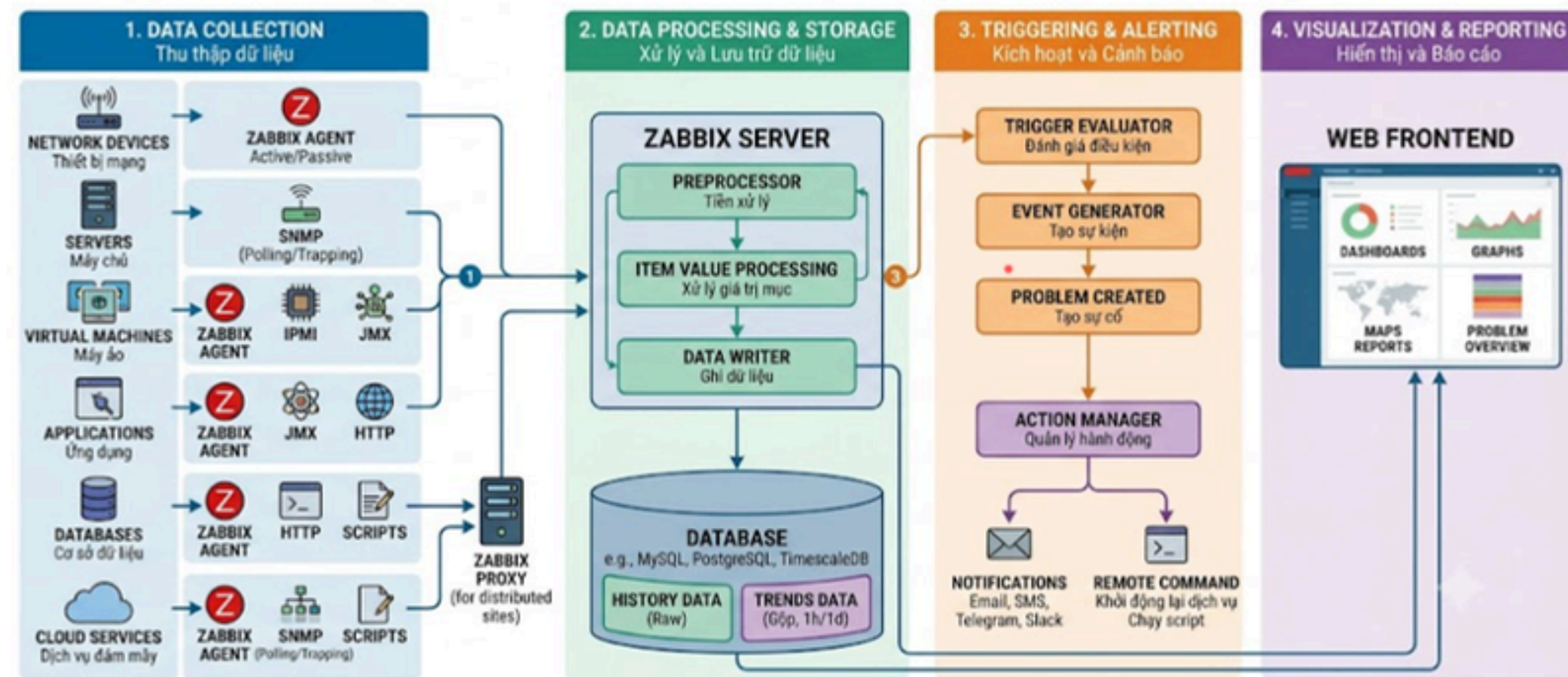
Thu thập & Truyền tải (Collect & Transfer):

- Sử dụng Zabbix Agent hoặc các giao thức (SNMP, IPMI, HTTP...) để lấy thông số thiết bị/dịch vụ.
- Gửi dữ liệu trực tiếp về Server hoặc trung chuyển qua Proxy.

Cảnh báo & Xử lý (Alert & Action):

- Tạo sự kiện ngay khi phát hiện bất thường.
- Tự động gửi thông báo (Email, Telegram, SMS) hoặc chạy script/khởi động lại dịch vụ.

LUỒNG HOẠT ĐỘNG CỦA ZABBIX ZABBIX OPERATIONAL FLOW



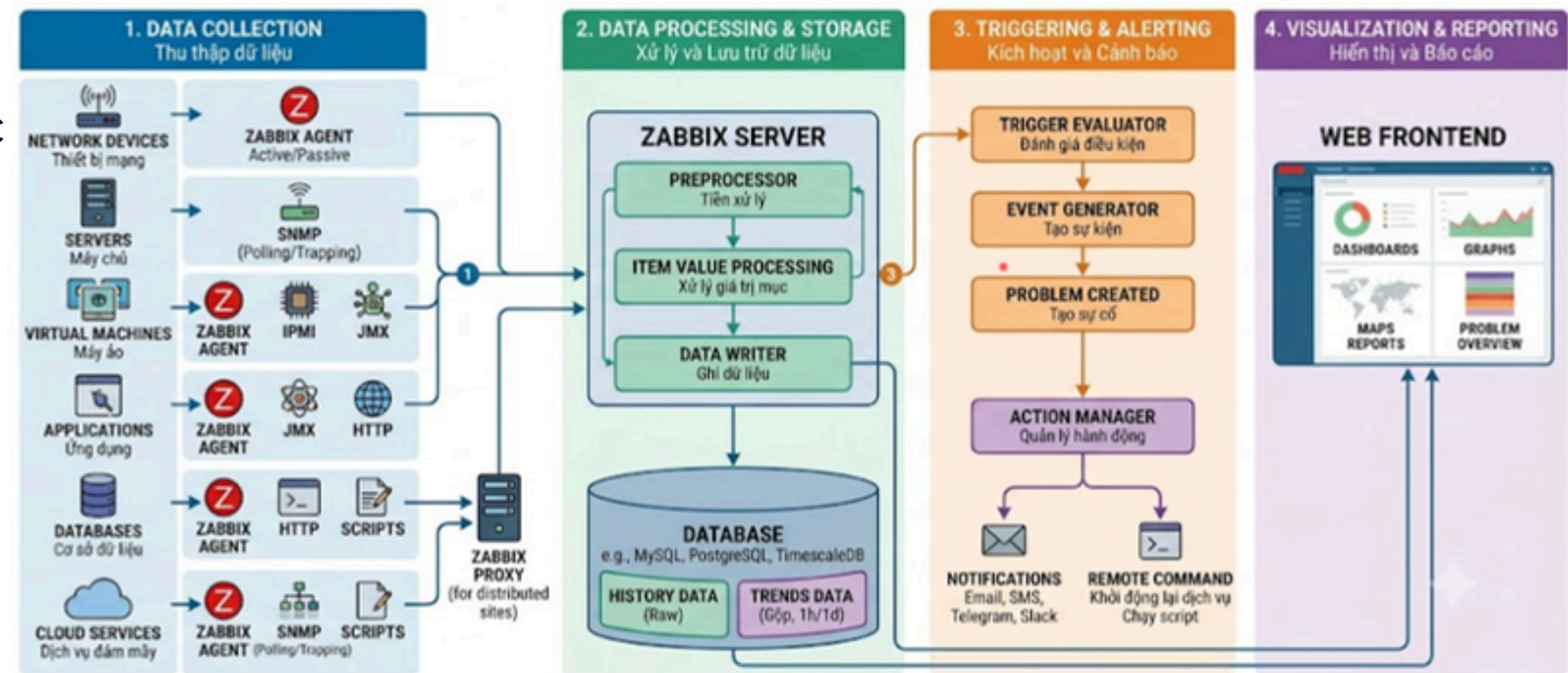
Nguyên lí hoạt động

Lưu trữ & Đánh giá (Store & Evaluate):

- Lưu trữ dữ liệu vào Database phục vụ phân tích lịch sử.
- Liên tục đối chiếu dữ liệu thu thập (Items) với các ngưỡng cảnh báo đã cài đặt (Triggers).

Trực quan hóa (Visualization): Hiển thị toàn cảnh trạng thái hệ thống, biểu đồ và báo cáo thông qua giao diện Web Frontend.

LUỒNG HOẠT ĐỘNG CỦA ZABBIX ZABBIX OPERATIONAL FLOW



Đánh giá **Zabbix**

✔ Ưu điểm

- ✔ All-in-one: Tích hợp sẵn quản trị, lưu trữ, cảnh báo và phân quyền (RBAC).
- ✔ Giám sát mạng mạnh: Hỗ trợ cho nhiều phương thức SNMP, IPMI cho phần cứng, Router, Switch.
- ✔ Dễ làm quen: Template khổng lồ và giao diện cấu hình trực quan.

⚠ Nhược điểm

- ✗ Khó quản lý môi trường Dynamic: Mô hình Host/Item/Trigger trở nên phức tạp hơn khi Container/Pod liên tục được tạo, xóa và thay đổi.
- ✗ Áp lực Database: Lượng dữ liệu monitoring lớn có thể làm tăng tải ghi lên Database và Disk I/O, đặc biệt khi hệ thống được mở rộng..





6

Khi nào nên dùng Prometheus + Grafana?

06. KHI NÀO NÊN DÙNG PROMETHEUS + GRAFANA?

SO SÁNH: PROMETHEUS + GRAFANA VS ZABBIX

Bảng so sánh 4 tiêu chí cốt lõi giữa 2 giải pháp giám sát hàng đầu

TIÊU CHÍ	 PROMETHEUS + GRAFANA	 ZABBIX
Hạ tầng phù hợp	Cloud-native Container (K8s/Docker), Microservices.	Truyền thống Network, Data Center, Server vật lý (Bare-metal).
Cơ chế thu thập	Pull-based: Prometheus chủ động kéo (scrape) metric định kỳ từ endpoints.	Linh hoạt Push/Pull: Đi kèm hệ thống Agent mạnh mẽ & hỗ trợ SNMP/IPMI.
Trực quan hóa	Tùy biến vô hạn: Giao diện hiển thị biểu đồ đẹp, động & linh hoạt nhờ Grafana.	Tích hợp sẵn: Giao diện dạng bảng/đồ thị cơ bản, ít linh hoạt hơn khi custom.
Alerting (Cảnh báo)	Tách biệt qua Alertmanager: Cấu hình linh hoạt dạng Code (phù hợp GitOps).	Tích hợp sẵn: Cấu hình quy tắc cảnh báo (Triggers) trực tiếp trên giao diện UI.

06. KHI NÀO NÊN DÙNG PROMETHEUS + GRAFANA?
KHI NÀO NÊN CHỌN GIẢI PHÁP NÀO?

Khung tiêu chí lựa chọn công cụ phù hợp với mô hình hạ tầng doanh nghiệp



PROMETHEUS + GRAFANA

Lựa chọn hàng đầu cho hệ thống hiện đại

- ✓ Hệ thống triển khai theo kiến trúc **Microservices, Cloud-native, Kubernetes / Docker.**
- ✓ Đòi hỏi **khả năng mở rộng linh hoạt** và cần **Dashboard trực quan, tùy biến sâu.**



ZABBIX

Giải pháp toàn diện cho hạ tầng truyền thống

- ✓ Giám sát **hạ tầng mạng doanh nghiệp hỗn hợp** (Router, Switch, Server vật lý / Bare-metal).
- ✓ Cần giải pháp **"Tất-cả-trong-một" (All-in-one)**, cấu hình tập trung trên UI mà không muốn tích hợp nhiều công cụ rời.



DEMO



8

Kết luận

TỔNG QUAN & GIÁ TRỊ CỐT LÕI

- **Tính bắt buộc của Monitoring:** đảm bảo tính ổn định, liên tục và tối ưu hiệu năng là yêu cầu bắt buộc trong kỷ nguyên công nghệ hiện đại.
- **Quy trình vận hành khép kín:**
 - **Prometheus:** đảm nhận thu thập, xử lý dữ liệu chuỗi thời gian gọn nhẹ, bền bỉ qua cơ chế Pull-based.
 - **Grafana:** chuyển hóa các con số thô khô khan thành hệ thống Dashboard động, trực quan và tùy biến vô hạn.
- **Thực nghiệm chứng minh:** kết hợp lý thuyết và kiểm thử ép tải thực tế (stress-ng) khẳng định sức mạnh và sự nhạy bén của giải pháp.



ĐỘT PHÁ TRONG VẬN HÀNH & VỊ THẾ CÔNG NGHỆ



- **"Combo Quốc Dân" trong môi trường hiện đại: giải pháp tiêu chuẩn không thể thay thế cho Cloud-native, Container và Microservices.**
- **Chuyển dịch tư duy quản trị:**
 - **Từ thế bị động: chờ xảy ra sự cố hoặc người dùng phản ánh mới xử lý.**
 - **Sang thế chủ động: phát hiện sớm dấu hiệu bất thường (rò rỉ bộ nhớ, cạn kiệt đĩa cứng) để can thiệp kịp thời.**
- **Giá trị thực tiễn: nâng cao năng lực quan sát, giảm thiểu tối đa thời gian gián đoạn dịch vụ và bảo vệ trải nghiệm người dùng cuối.**

